Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-214Б-24

Студент: Ходырев Д.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.12.25

Москва, 2024

Постановка задачи

Вариант 13.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Наложить K раз фильтр, использующий матрицу свертки, на матрицу, состоящую из вещественных чисел. Размер окна задается пользователем

Общий метод и алгоритм решения

Использованные системные вызовы:

- *int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);* – создаёт новый поток, который начинает выполнение функции start_routine с аргументом arg.

- *int pthread_join(pthread_t thread, void **retval);* – ожидает завершения указанного потока и возвращает его результат.

- *int clock_gettime(clockid_t clockid, struct timespec *tp);* – получает текущее время, указанное идентификатором *clock_id* и записывает его в структуру *struct timespec*, на которую указывает параметр *tp*.

План реализации:

1. Последовательная версия

- Читаем матрицу из файла или генерируем случайно
- Читаем ядро свёртки из файла или задаём в коде
- Применяем свёртку K раз:
    - Для каждого внутреннего элемента вычисляем взвешенную сумму по окну
    - Учитываем края (можно использовать padding: zero, mirror, replicate)
- Сохраняем результат

2. Параллельная версия (POSIX Threads)

- Ограничиваем максимальное количество потоков через аргумент командной строки (-t N)
- Распараллеливаем обработку по строкам матрицы:
    - Каждый поток обрабатывает свой блок строк

- Используем мьютексы или барьеры, если K > 1 (чтобы следующая итерация свёртки начиналась после завершения предыдущей всеми потоками)

- Замеряем время выполнения

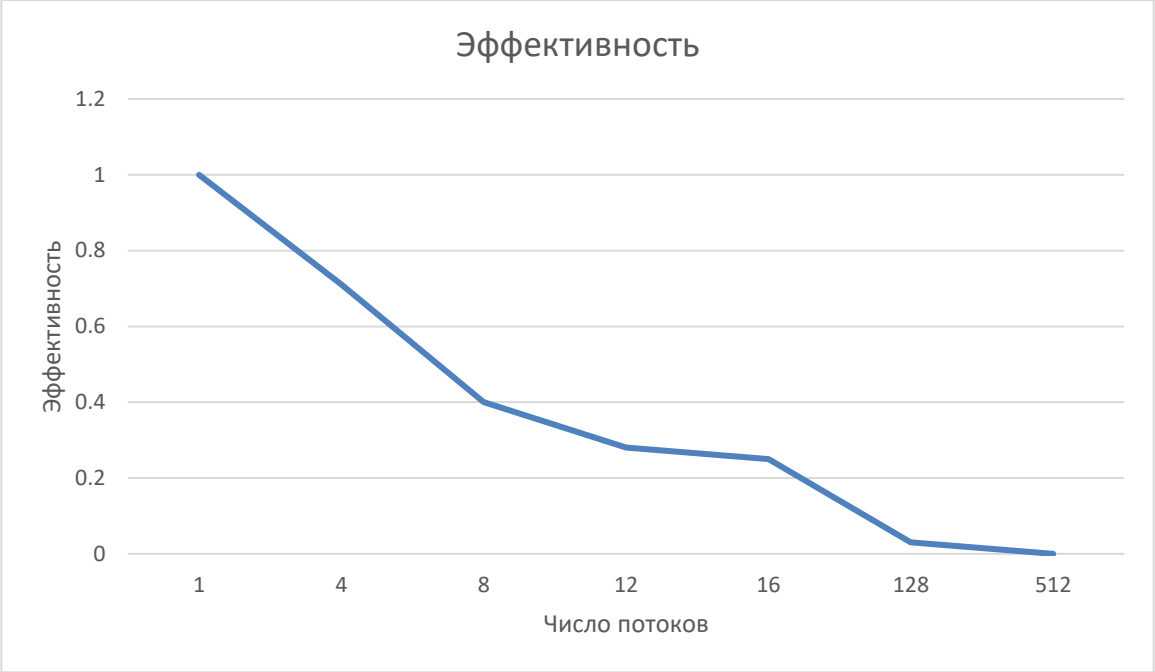
3. Измерения и отчёт

- Запускаем для разных количеств потоков
- Считаем ускорение и эффективность
- Строим таблицу и графики

Анализ ускорения и эффективности

Число потоков	Время выполнения	Ускорение	Эффективность
1	150	1	1
4	52,81	2,84	0,71
8	46,71	3,23	0,4
12	45,43	3,31	0,28
16	37,46	4,04	0,25
128	45,75	3,21	0,03
512	74,25	1,92	0,00

Самое высокое ускорение наблюдалось при использовании 16 потоков. В этой точке ускорение составило 4. До этого значения ускорение увеличивалось, после – стало падать.



Высокая эффективность наблюдалась до 8 потоков. При большем количестве потоков планировщик перегружается и эффективность падает практически до нуля.

Код программы

convolution.h

```
#pragma once

#include <stddef.h>
#include <pthread.h>

typedef struct {
    float** data;
    int rows;
    int cols;
} Matrix;

// Структура для параметров свёртки
typedef struct {
    Matrix* input;
    Matrix* kernel;
    Matrix* output;
    int iterations;
    int start_row;
    int end_row;
    pthread_barrier_t* barrier;
} ConvolutionTask;

Matrix* create_matrix(int rows, int cols);
void free_matrix(Matrix* m);
Matrix* read_matrix_from_file(const char* filename);
void write_matrix_to_file(const char* filename, Matrix* m);
Matrix* apply_convolution_sequential(Matrix* input, Matrix* kernel, int iterations);
Matrix* apply_convolution_parallel(Matrix* input, Matrix* kernel, int iterations, int num_threads);
void print_matrix(Matrix* m);
```

convolution.c

```
#include "convolution.h"
#include "utils.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <stdatomic.h>

Matrix* create_matrix(int rows, int cols) {
    Matrix* m = (Matrix*)malloc(sizeof(Matrix));
    m->rows = rows;
    m->cols = cols;
    m->data = (float**)malloc(rows * sizeof(float*));
    for (int i = 0; i < rows; i++) {
        m->data[i] = (float*)calloc(cols, sizeof(float));
    }
    return m;
}

void free_matrix(Matrix* m) {
    if (!m) return;
    for (int i = 0; i < m->rows; i++) {
        free(m->data[i]);
    }
    free(m->data);
    free(m);
}

Matrix* read_matrix_from_file(const char* filename) {
    FILE* file = fopen(filename, "r");
    if (!file) {
```

```
        perror("Failed to open file");
        return NULL;
    }

    int rows, cols;
    fscanf(file, "%d %d", &rows, &cols);

    Matrix* m = create_matrix(rows, cols);

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            fscanf(file, "%f", &m->data[i][j]);
        }
    }

    fclose(file);
    return m;
}

void write_matrix_to_file(const char* filename, Matrix* m) {
    FILE* file = fopen(filename, "w");
    if (!file) {
        perror("Failed to open file for writing");
        return;
    }

    fprintf(file, "%d %d\n", m->rows, m->cols);

    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
            fprintf(file, "%.6f ", m->data[i][j]);
        }
        fprintf(file, "\n");
    }

    fclose(file);
}

void print_matrix(Matrix* m) {
    if (!m) return;
    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
            printf("%.2f ", m->data[i][j]);
        }
        printf("\n");
    }
}

void apply_convolution_partial(Matrix* input, Matrix* kernel, Matrix* output, int start_row, int end_row) {
    int k_half = kernel->rows / 2;
    int k_size = kernel->rows;

    for (int i = start_row; i < end_row; i++) {
        for (int j = 0; j < input->cols; j++) {
            float sum = 0.0f;

            for (int ki = 0; ki < k_size; ki++) {
                for (int kj = 0; kj < k_size; kj++) {
                    int input_i = i + ki - k_half;
                    int input_j = j + kj - k_half;

                    if (input_i >= 0 && input_i < input->rows &&
                        input_j >= 0 && input_j < input->cols) {
                        sum += input->data[input_i][input_j] * kernel->data[ki][kj];
                    }
                }
```

```
            }

            output->data[i][j] = sum;
        }
    }
}

void* convolution_thread_func(void* arg) {
    ConvolutionTask* task = (ConvolutionTask*)arg;
    Matrix* temp1 = task->input;
    Matrix* temp2 = task->output;

    for (int iter = 0; iter < task->iterations; iter++) {
        apply_convolution_partial(temp1, task->kernel, temp2,
                                  task->start_row, task->end_row);

        pthread_barrier_wait(task->barrier);

        Matrix* temp = temp1;
        temp1 = temp2;
        temp2 = temp;
    }

    return NULL;
}

Matrix* apply_convolution_sequential(Matrix* input, Matrix* kernel, int iterations) {
    Matrix* result = create_matrix(input->rows, input->cols);
    Matrix* buffer1 = create_matrix(input->rows, input->cols);
    Matrix* buffer2 = create_matrix(input->rows, input->cols);

    for (int i = 0; i < input->rows; i++) {
        memcpy(buffer1->data[i], input->data[i], input->cols * sizeof(float));
    }

    for (int iter = 0; iter < iterations; iter++) {
        apply_convolution_partial(buffer1, kernel, buffer2, 0, input->rows);

        Matrix* temp = buffer1;
        buffer1 = buffer2;
        buffer2 = temp;
    }

    for (int i = 0; i < input->rows; i++) {
        memcpy(result->data[i], buffer1->data[i], input->cols * sizeof(float));
    }

    free_matrix(buffer1);
    free_matrix(buffer2);

    return result;
}

Matrix* apply_convolution_parallel(Matrix* input, Matrix* kernel, int iterations, int num_threads) {
    pthread_t* threads = (pthread_t*)malloc(num_threads * sizeof(pthread_t));
    ConvolutionTask* tasks = (ConvolutionTask*)malloc(num_threads * sizeof(ConvolutionTask));

    pthread_barrier_t barrier;
    pthread_barrier_init(&barrier, NULL, num_threads);

    Matrix* buffer1 = create_matrix(input->rows, input->cols);
    Matrix* buffer2 = create_matrix(input->rows, input->cols);

    for (int i = 0; i < input->rows; i++) {
        memcpy(buffer1->data[i], input->data[i], input->cols * sizeof(float));
    }
```

```
    int rows_per_thread = input->rows / num_threads;
    int extra_rows = input->rows % num_threads;

    int current_row = 0;
    for (int i = 0; i < num_threads; i++) {
        tasks[i].input = buffer1;
        tasks[i].output = buffer2;
        tasks[i].kernel = kernel;
        tasks[i].iterations = iterations;
        tasks[i].barrier = &barrier;

        tasks[i].start_row = current_row;
        tasks[i].end_row = current_row + rows_per_thread + (i < extra_rows ? 1 : 0);
        current_row = tasks[i].end_row;

        pthread_create(&threads[i], NULL, convolution_thread_func, &tasks[i]);
    }

    for (int i = 0; i < num_threads; i++) {
        pthread_join(threads[i], NULL);
    }

    Matrix* result = create_matrix(input->rows, input->cols);
    for (int i = 0; i < input->rows; i++) {
        memcpy(result->data[i], buffer1->data[i], input->cols * sizeof(float));
    }

    pthread_barrier_destroy(&barrier);
    free_matrix(buffer1);
    free_matrix(buffer2);
    free(threads);
    free(tasks);

    return result;
}
```

utils.h

```
#pragma once

#include <time.h>

typedef struct {
    struct timespec start;
    struct timespec end;
} Timer;

void timer_start(Timer* timer);
void timer_stop(Timer* timer);
double timer_elapsed_ms(Timer* timer);

void generate_random_matrix(const char* filename, int rows, int cols);
void generate_kernel(const char* filename, int size);
```

utils.c

```
#include "utils.h"
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void timer_start(Timer* timer) {
    clock_gettime(CLOCK_MONOTONIC, &timer->start);
}

void timer_stop(Timer* timer) {
```

```
    clock_gettime(CLOCK_MONOTONIC, &timer->end);
}

double timer_elapsed_ms(Timer* timer) {
    double start_ms = timer->start.tv_sec * 1000.0 + timer->start.tv_nsec / 1000000.0;
    double end_ms = timer->end.tv_sec * 1000.0 + timer->end.tv_nsec / 1000000.0;
    return end_ms - start_ms;
}

void generate_random_matrix(const char* filename, int rows, int cols) {
    FILE* file = fopen(filename, "w");
    if (!file) {
        perror("Failed to create matrix file");
        return;
    }

    fprintf(file, "%d %d\n", rows, cols);
    srand(time(NULL));

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            fprintf(file, "%.6f ", (float)rand() / RAND_MAX * 100.0f);
        }
        fprintf(file, "\n");
    }

    fclose(file);
    printf("Generated matrix %dx%d to %s\n", rows, cols, filename);
}

void generate_kernel(const char* filename, int size) {
    FILE* file = fopen(filename, "w");
    if (!file) {
        perror("Failed to create kernel file");
        return;
    }

    fprintf(file, "%d %d\n", size, size);

    float value = 1.0f / (size * size);

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            fprintf(file, "%.6f ", value);
        }
        fprintf(file, "\n");
    }

    fclose(file);
    printf("Generated %dx%d kernel to %s\n", size, size, filename);
}
```

main.c

```
#include "convolution.h"
#include "utils.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/sysinfo.h>

void print_usage(const char* prog_name) {
    printf("Usage: %s [options]\n", prog_name);
    printf("Options:\n");
    printf("  -i <file>    Input matrix file\n");
    printf("  -k <file>    Kernel file\n");
```

```
    printf("  -o <file>    Output matrix file\n");
    printf("  -t <N>       Number of threads (default: 1)\n");
    printf("  -iter <K>    Number of iterations (default: 1)\n");
    printf("  -seq         Run sequential version only\n");
    printf("  -par         Run parallel version only\n");
    printf("  -gen <size>  Generate random matrix of given size\n");
    printf("  -help        Show this help\n");
}

int main(int argc, char* argv[]) {
    char* input_file = NULL;
    char* kernel_file = NULL;
    char* output_file = "output.txt";
    int num_threads = 1;
    int iterations = 1;
    int run_seq = 0;
    int run_par = 0;
    int generate_size = 0;

    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-i") == 0 && i + 1 < argc) {
            input_file = argv[++i];
        } else if (strcmp(argv[i], "-k") == 0 && i + 1 < argc) {
            kernel_file = argv[++i];
        } else if (strcmp(argv[i], "-o") == 0 && i + 1 < argc) {
            output_file = argv[++i];
        } else if (strcmp(argv[i], "-t") == 0 && i + 1 < argc) {
            num_threads = atoi(argv[++i]);
        } else if (strcmp(argv[i], "-iter") == 0 && i + 1 < argc) {
            iterations = atoi(argv[++i]);
        } else if (strcmp(argv[i], "-seq") == 0) {
            run_seq = 1;
        } else if (strcmp(argv[i], "-par") == 0) {
            run_par = 1;
        } else if (strcmp(argv[i], "-gen") == 0 && i + 1 < argc) {
            generate_size = atoi(argv[++i]);
        } else if (strcmp(argv[i], "-help") == 0) {
            print_usage(argv[0]);
            return 0;
        }
    }

    if (generate_size > 0) {
        char matrix_file[100];
        char kernel_file[100];

        sprintf(matrix_file, "matrix_%dx%d.txt", generate_size, generate_size);
        sprintf(kernel_file, "kernel_3x3.txt");

        generate_random_matrix(matrix_file, generate_size, generate_size);
        generate_kernel(kernel_file, 3);

        printf("Test data generated. Run with:\n");
        printf("  %s -i %s -k %s -t 4 -iter 3\n", argv[0], matrix_file, kernel_file);
        return 0;
    }

    if (!input_file || !kernel_file) {
        print_usage(argv[0]);
        return 1;
    }

    Matrix* input = read_matrix_from_file(input_file);
    Matrix* kernel = read_matrix_from_file(kernel_file);

    if (!input || !kernel) {
```

```
        fprintf(stderr, "Failed to read input files\n");
        return 1;
    }

    printf("Matrix: %dx%d, Kernel: %dx%d, Threads: %d, Iterations: %d\n",
           input->rows, input->cols, kernel->rows, kernel->cols, num_threads, iterations);

    if (!run_seq && !run_par) {
        run_seq = 1;
        run_par = 1;
    }

    Matrix* result_seq = NULL;
    Matrix* result_par = NULL;
    Timer timer;
    double time_seq = 0, time_par = 0;

    if (run_seq) {
        printf("\n=== Sequential version ===\n");
        timer_start(&timer);
        result_seq = apply_convolution_sequential(input, kernel, iterations);
        timer_stop(&timer);
        time_seq = timer_elapsed_ms(&timer);
        printf("Time: %.2f ms\n", time_seq);
    }

    if (run_par) {
        printf("\n=== Parallel version (%d threads) ===\n", num_threads);
        timer_start(&timer);
        result_par = apply_convolution_parallel(input, kernel, iterations, num_threads);
        timer_stop(&timer);
        time_par = timer_elapsed_ms(&timer);
        printf("Time: %.2f ms\n", time_par);

        if (run_seq) {
            double speedup = time_seq / time_par;
            double efficiency = speedup / num_threads;
            printf("Speedup: %.2f\n", speedup);
            printf("Efficiency: %.2f\n", efficiency);
        }
    }

    if (result_par) {
        write_matrix_to_file(output_file, result_par);
        printf("Result written to %s\n", output_file);
    } else if (result_seq) {
        write_matrix_to_file(output_file, result_seq);
        printf("Result written to %s\n", output_file);
    }

    free_matrix(input);
    free_matrix(kernel);
    free_matrix(result_seq);
    free_matrix(result_par);

    return 0;
}
```

Протокол работы программы

```
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 1 -iter 10
Matrix: 1000x1000, Kernel: 3x3, Threads: 1, Iterations: 10

=== Sequential version ===
Time: 150.35 ms

=== Parallel version (1 threads) ===
Time: 150.81 ms
Speedup: 1.0
Efficiency: 1.0
Result written to output.txt
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 4 -iter 10
Matrix: 1000x1000, Kernel: 3x3, Threads: 4, Iterations: 10

=== Sequential version ===
Time: 151.93 ms

=== Parallel version (4 threads) ===
Time: 52.81 ms
Speedup: 2.84
Efficiency: 0.71
Result written to output.txt
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 8 -iter 10
Matrix: 1000x1000, Kernel: 3x3, Threads: 8, Iterations: 10

=== Sequential version ===
Time: 149.44 ms

=== Parallel version (8 threads) ===
Time: 46.71 ms
Speedup: 3.23
Efficiency: 0.40
Result written to output.txt
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 12 -iter 10
Matrix: 1000x1000, Kernel: 3x3, Threads: 12, Iterations: 10

=== Sequential version ===
Time: 150.70 ms

=== Parallel version (12 threads) ===
Time: 45.43 ms
Speedup: 3.31
Efficiency: 0.28
Result written to output.txt
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 16 -iter 10
Matrix: 1000x1000, Kernel: 3x3, Threads: 16, Iterations: 10

=== Sequential version ===
Time: 151.30 ms

=== Parallel version (16 threads) ===
Time: 37.46 ms
Speedup: 4.04
```

Efficiency: 0.25
Result written to output.txt
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 128 -iter 10Matrix: 1000x1000, Kernel: 3x3, Threads: 128, Iterations: 10

=== Sequential version ===
Time: 157.32 ms

=== Parallel version (128 threads) ===
Time: 45.75 ms
Speedup: 3.21
Efficiency: 0.03
Result written to output.txt
d_khod@D-Khod-Laptop:/mnt/c/Users/g66xq/Уник/Лабы_по_ОС/mai_OS_labs/laba2/src$ ./convolution -i matrix_1000x1000.txt -k kernel_3x3.txt -t 512 -iter 10
Matrix: 1000x1000, Kernel: 3x3, Threads: 512, Iterations: 10

=== Sequential version ===
Time: 158.45 ms

=== Parallel version (512 threads) ===
Time: 74.25 ms
Speedup: 1.92
Efficiency: 0.00
Result written to output.txt

Команда strace
execve("./convolution", ["./convolution", "-i", "matrix_1000x1000.txt", "-k", "kernel_3x3.txt", "-t", "16", "-iter", "10"], 0x7ffd266c66a0 /* 28 vars */) = 0
brk(NULL) = 0x5dc95e0d4000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x78ab78b34000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=31235, ...}) = 0
mmap(NULL, 31235, PROT_READ, MAP_PRIVATE, 3, 0) = 0x78ab78b2c000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x78ab78800000
mmap(0x78ab78828000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x78ab78828000
mmap(0x78ab789b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x78ab789b0000
mmap(0x78ab789ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x78ab789ff000
mmap(0x78ab78a05000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x78ab78a05000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x78ab78b29000
arch_prctl(ARCH_SET_FS, 0x78ab78b29740) = 0
set_tid_address(0x78ab78b29a10) = 14420
set_robust_list(0x78ab78b29a20, 24) = 0
rseq(0x78ab78b2a060, 0x20, 0, 0x53053053) = 0
mprotect(0x78ab789ff000, 16384, PROT_READ) = 0

```
mprotect(0x5dc95cf15000, 4096, PROT_READ) = 0
mprotect(0x78ab78b6c000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x78ab78b2c000, 31235)           = 0
getrandom("\x8d\x51\x4c\x6a\x94\x98\xa8\x39", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5dc95e0d4000
brk(0x5dc95e0f5000)                     = 0x5dc95e0f5000
openat(AT_FDCWD, "matrix_1000x1000.txt", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0777, st_size=9900711, ...}) = 0
read(3, "1000 1000\n35.842136 88.958679 44"..., 4096) = 4096
brk(0x5dc95e116000)                     = 0x5dc95e116000
brk(0x5dc95e137000)                     = 0x5dc95e137000
brk(0x5dc95e158000)                     = 0x5dc95e158000
brk(0x5dc95e179000)                     = 0x5dc95e179000
brk(0x5dc95e19a000)                     = 0x5dc95e19a000
brk(0x5dc95e1bb000)                     = 0x5dc95e1bb000
brk(0x5dc95e1dc000)                     = 0x5dc95e1dc000
brk(0x5dc95e1fd000)                     = 0x5dc95e1fd000
brk(0x5dc95e21e000)                     = 0x5dc95e21e000
brk(0x5dc95e23f000)                     = 0x5dc95e23f000
brk(0x5dc95e260000)                     = 0x5dc95e260000
brk(0x5dc95e281000)                     = 0x5dc95e281000
brk(0x5dc95e2a2000)                     = 0x5dc95e2a2000
brk(0x5dc95e2c3000)                     = 0x5dc95e2c3000
brk(0x5dc95e2e4000)                     = 0x5dc95e2e4000
brk(0x5dc95e305000)                     = 0x5dc95e305000
brk(0x5dc95e326000)                     = 0x5dc95e326000
brk(0x5dc95e347000)                     = 0x5dc95e347000
brk(0x5dc95e368000)                     = 0x5dc95e368000
brk(0x5dc95e389000)                     = 0x5dc95e389000
brk(0x5dc95e3aa000)                     = 0x5dc95e3aa000
brk(0x5dc95e3cb000)                     = 0x5dc95e3cb000
brk(0x5dc95e3ec000)                     = 0x5dc95e3ec000
brk(0x5dc95e40d000)                     = 0x5dc95e40d000
brk(0x5dc95e42e000)                     = 0x5dc95e42e000
brk(0x5dc95e44f000)                     = 0x5dc95e44f000
brk(0x5dc95e470000)                     = 0x5dc95e470000
brk(0x5dc95e491000)                     = 0x5dc95e491000
brk(0x5dc95e4b2000)                     = 0x5dc95e4b2000
read(3, ".297262 92.905792 43.772121 91.5"..., 4096) = 4096
...OOOOчень много вызовов...
read(3, "14273 35.079014 55.750362 42.897"..., 4096) = 4096
read(3, "66 97.040764 86.248344 0.312179 "..., 4096) = 679
close(3)                                = 0
openat(AT_FDCWD, "kernel_3x3.txt", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0777, st_size=88, ...}) = 0
read(3, "3 3\n0.111111 0.111111 0.111111 \n"..., 4096) = 88
close(3)                                = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
write(1, "Matrix: 1000x1000, Kernel: 3x3, "..., 60) = 60
write(1, "\n", 1)                       = 1
write(1, "=== Sequential version ===\n", 27) = 27
brk(0x5dc95e4d3000)                     = 0x5dc95e4d3000
...Много таких вызовов...
brk(0x5dc95ec7b000)                     = 0x5dc95ec7b000
brk(0x5dc95e8a3000)                     = 0x5dc95e8a3000
write(1, "Time: 197.82 ms\n", 16)       = 16
write(1, "\n=== Parallel version (16 thread"..., 39) = 39
brk(0x5dc95e8c4000)                     = 0x5dc95e8c4000
brk(0x5dc95e8e5000)                     = 0x5dc95e8e5000
```

...Много таких вызовов...
brk(0x5dc95f01d000) = 0x5dc95f01d000
brk(0x5dc95f03e000) = 0x5dc95f03e000
rt_sigaction(SIGRT_1, {sa_handler=0x78ab78899530, sa_mask=[], sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x78ab78845330}, NULL, 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab77fff000
mprotect(0x78ab78000000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x78ab787ff990, parent_tid=0x78ab787ff990, exit_signal=0, stack=0x78ab77fff000, stack_size=0x7fff80, tls=0x78ab787ff6c0} => {parent_tid=[14438]}, 88) = 14438
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab777fe000
mprotect(0x78ab777ff000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x78ab77ffe990, parent_tid=0x78ab77ffe990, exit_signal=0, stack=0x78ab777fe000, stack_size=0x7fff80, tls=0x78ab77ffe6c0} => {parent_tid=[14439]}, 88) = 14439
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab76ffd000
mprotect(0x78ab76ffe000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x78ab777fd990, parent_tid=0x78ab777fd990, exit_signal=0, stack=0x78ab76ffd000, stack_size=0x7fff80, tls=0x78ab777fd6c0} => {parent_tid=[14440]}, 88) = 14440
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab767fc000
mprotect(0x78ab767fd000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x78ab76ffc990, parent_tid=0x78ab76ffc990, exit_signal=0, stack=0x78ab767fc000, stack_size=0x7fff80, tls=0x78ab76ffc6c0} => {parent_tid=[14441]}, 88) = 14441
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab75ffb000
mprotect(0x78ab75ffc000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x78ab767fb990, parent_tid=0x78ab767fb990, exit_signal=0, stack=0x78ab75ffb000, stack_size=0x7fff80, tls=0x78ab767fb6c0} => {parent_tid=[14442]}, 88) = 14442
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab757fa000
mprotect(0x78ab757fb000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x78ab75ffa990, parent_tid=0x78ab75ffa990, exit_signal=0, stack=0x78ab757fa000, stack_size=0x7fff80, tls=0x78ab75ffa6c0} => {parent_tid=[14443]}, 88) = 14443

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab74ff9000
mprotect(0x78ab74ffa000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab757f9990, parent_tid=0x78ab757f9990, exit_signal=0, stack=0x78ab74ff9000, stack_size=0x7fff80, tls=0x78ab757f96c0} => {parent_tid=[14444]}, 88) = 14444
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab747f8000
mprotect(0x78ab747f9000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab74ff8990, parent_tid=0x78ab74ff8990, exit_signal=0, stack=0x78ab747f8000, stack_size=0x7fff80, tls=0x78ab74ff86c0} => {parent_tid=[14445]}, 88) = 14445
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab73ff7000
mprotect(0x78ab73ff8000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab747f7990, parent_tid=0x78ab747f7990, exit_signal=0, stack=0x78ab73ff7000, stack_size=0x7fff80, tls=0x78ab747f76c0} => {parent_tid=[14446]}, 88) = 14446
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab737f6000
mprotect(0x78ab737f7000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab73ff6990, parent_tid=0x78ab73ff6990, exit_signal=0, stack=0x78ab737f6000, stack_size=0x7fff80, tls=0x78ab73ff66c0} => {parent_tid=[14447]}, 88) = 14447
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab72ff5000
mprotect(0x78ab72ff6000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab737f5990, parent_tid=0x78ab737f5990, exit_signal=0, stack=0x78ab72ff5000, stack_size=0x7fff80, tls=0x78ab737f56c0} => {parent_tid=[14448]}, 88) = 14448
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab727f4000
mprotect(0x78ab727f5000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab72ff4990, parent_tid=0x78ab72ff4990, exit_signal=0, stack=0x78ab727f4000, stack_size=0x7fff80, tls=0x78ab72ff46c0} => {parent_tid=[14449]}, 88) = 14449
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab71ff3000
mprotect(0x78ab71ff4000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab727f3990, parent_tid=0x78ab727f3990, exit_signal=0, stack=0x78ab71ff3000, stack_size=0x7fff80, tls=0x78ab727f36c0} => {parent_tid=[14450]}, 88) = 14450
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab717f2000
mprotect(0x78ab717f3000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab71ff2990, parent_tid=0x78ab71ff2990, exit_signal=0, stack=0x78ab717f2000, stack_size=0x7fff80, tls=0x78ab71ff26c0} => {parent_tid=[14451]}, 88) = 14451
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab70ff1000
mprotect(0x78ab70ff2000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab717f1990, parent_tid=0x78ab717f1990, exit_signal=0, stack=0x78ab70ff1000, stack_size=0x7fff80, tls=0x78ab717f16c0} => {parent_tid=[14452]}, 88) = 14452
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x78ab707f0000
mprotect(0x78ab707f1000, 8388608, PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x78ab70ff0990, parent_tid=0x78ab70ff0990, exit_signal=0, stack=0x78ab707f0000, stack_size=0x7fff80, tls=0x78ab70ff06c0} => {parent_tid=[14453]}, 88) = 14453
rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
futex(0x78ab787ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 14438, NULL, FUTEX_BITSET_MATCH_ANY) = 0
munmap(0x78ab77fff000, 8392704) = 0
munmap(0x78ab777fe000, 8392704) = 0
munmap(0x78ab76ffd000, 8392704) = 0
munmap(0x78ab767fc000, 8392704) = 0
munmap(0x78ab75ffb000, 8392704) = 0
munmap(0x78ab757fa000, 8392704) = 0
munmap(0x78ab74ff9000, 8392704) = 0
munmap(0x78ab747f8000, 8392704) = 0
munmap(0x78ab73ff7000, 8392704) = 0
munmap(0x78ab737f6000, 8392704) = 0
munmap(0x78ab72ff5000, 8392704) = 0
munmap(0x78ab727f4000, 8392704) = 0
brk(0x5dc95f05f000) = 0x5dc95f05f000
brk(0x5dc95f080000) = 0x5dc95f080000
...
brk(0x5dc95f41c000) = 0x5dc95f41c000
write(1, "Time: 196.57 ms\n", 16) = 16
write(1, "Speedup: 1.01\n", 14) = 14
write(1, "Efficiency: 0.06\n", 17) = 17
openat(AT_FDCWD, "output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
fstat(3, {st_mode=S_IFREG|0777, st_size=0, ...}) = 0
write(3, "1000 1000\n4.720781 8.890074 12.1"..., 4096) = 4096
write(3, "5 15.185316 15.577061 15.786007 "..., 4096) = 4096
...Опять очень много write...
write(3, "5.102266 14.573035 14.197713 14."..., 4096) = 4096
write(3, "89 13.396897 13.584679 14.022306"..., 2662) = 2662

```
close(3)                                = 0
write(1, "Result written to output.txt\n", 29) = 29
brk(0x5dc95f052000)                     = 0x5dc95f052000
brk(0x5dc95f050000)                     = 0x5dc95f050000
exit_group(0)                           = ?
+++ exited with 0 +++
```

Вывод

В ходе выполнения лабораторной работы были успешно изучены и применены основные системные вызовы для работы с потоками в ОС Linux.